SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 2

Name of Student : Chatharaboina sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task 1: Cleaning Sensor Data

❖ Scenario:You are cleaning IoT sensor data where negative values are invalid.
❖ Task:Use Gemini in Colab to generate a function that filters out all negative numbers from a list.
❖ Expected Output:
➢ Before/after list
➢ Screenshot of Colab execution

Prompt:-
Write a Python function to return a new list containing only values ≥ 0 (remove negatives).Create a function to clean IoT sensor readings by filtering non-negativevalues without changing the original list.

Code:-

lab2.py > ...

```
#Write a Python function to return a new list containing only values ≥ 0 (remove negatives).
#Create a function to clean IoT sensor readings by filtering non-negative
# values without changing the original list.
def filter_non_negative(sensor_readings):
    return [reading for reading in sensor_readings if reading >= 0]
# user in input manner
try:
    input_readings = input("Enter sensor readings separated by spaces: ")
    sensor_readings = list(map(float, input_readings.split()))
    cleaned_readings = filter_non_negative(sensor_readings)
    print("Original readings:", sensor_readings)
    print("Cleaned readings (non-negative):", cleaned_readings)
except ValueError:
    print("Invalid input. Please enter numeric values only.")

```

Output:-

```
Enter sensor readings separated by spaces: 23 -2 34 0 34 65
Original readings: [23.0, -2.0, 34.0, 0.0, 34.0, 65.0]
Cleaned readings (non-negative): [23.0, 34.0, 0.0, 34.0, 65.0]
```

Justification:-
This function uses a list comprehension to create a new list that includes only the non-negative values from the original list. It does not modify the original

list,ensuring that the raw sensor data remains intact for any further analysis or processing.

Task 2: String Character Analysis

❖ Scenario:

You are building a text-analysis feature.

❖ Task:

Use Gemini to generate a Python function that counts vowels, consonants, and digits in a string.

❖ Expected Output:

➢ Working function

➢ Sample inputs and outputs

Prompt:-

Write a Python function to count vowels, consonants, and digits in a string (case-insensitive). Create a simple function that checks a text and returns the number of vowels, consonants, and numbers

Code:-

```
#----Task 2----
#Write a Python function to count vowels, consonants, and digits in a string (case-insensitive).
#Create a simple function that checks a text and returns the number of vowels, consonants, and numbers
def count_characters(text):
    vowels = "aeiouAEIOU"
    vowel_count = sum(1 for char in text if char in vowels)
    consonant_count = sum(1 for char in text if char.isalpha() and char not in vowels)
    digit_count = sum(1 for char in text if char.isdigit())
    return vowel_count, consonant_count, digit_count
# user input manner
input_text = input("Enter a string to analyze: ")
vowels, consonants, digits = count_characters(input_text)
print(f"Vowels: {vowels}, Consonants: {consonants}, Digits: {digits}")
```

Output:-

```
Input text: Hello World! 123
Vowels: 3, Consonants: 7, Digits: 3
```

Justification:-This function iterates through each character in the input string, checking if it is a vowel, consonant, or digit, and increments the respective

counters accordingly. It treats upper and lower case letters the same by including both in the vowels string and using isalpha() for consonants.

Task 3: Palindrome Check – Tool Comparison

❖ Scenario:

You must decide which AI tool is clearer for string logic.

❖ Task:

Generate a palindrome-checking function using Gemini and Copilot, then

compare the results.

❖ Expected Output:

➢ Side-by-side code comparison

➢ Observations on clarity and structure

Prompt:-

write a Python function is palindrome to check if a string is the same forwards and backwards ignoring spaces and case.

Create function that returns True/False to validate a palindrome string without considering spaces and uppercase/lowercase

Code:-

```
#--Task 3--
#rite a Python function is_palindrome to check if a string is the same forwards and backwards ignoring spaces and 
#Create function that returns True/False to validate
# a palindrome string without considering spaces and uppercase/lowercase.
def is_palindrome(s):
    cleaned_string = ''.join(char.lower() for char in s if char.isalnum())
    return cleaned_string == cleaned_string[::-1]
# user input manner
input_string = input("Enter a string to check if it is a palindrome: ")
if is_palindrome(input_string):
    print(f'"{input_string}" is a palindrome.')
else:
    print(f'"{input_string}" is not a palindrome.')
```

Output:-

```
Enter a string to check if it is a palindrome: amanaplanacanalpanama
"amanaplanacanalpanama" is a palindrome.
```

Justification:-

This function is simple and readable because it breaks down the problem into clear steps: cleaning the string by removing non-alphanumeric characters and converting to lowercase, then checking if the cleaned string is equal to its reverse. This makes it easy for beginners to understand how palindromes work.

Task 4: Code Explanation Using AI

❖ Scenario:

You are reviewing unfamiliar code written by another developer.

❖ Task:

Ask Gemini to explain a Python function (prime check OR palindrome check) line by line.

SR UNIVERSITY

❖ Expected Output:

➢ Code snippet
➢ AI explanation
➢ Student comments on understanding

Prompt:-

WritePython function is_prime(n) that checks whether a number is prime and explain it line by line in simple language.Write a Python function is_palindrome(text) that checks whether a string is a palindrome (ignore spaces and case) and explain it line by line for beginners

Code:-

```
#--Task 4--
#WritePython function is_prime(n) that checks whether a number is prime and explain it line by line in simple language.
#Write a Python function is_palindrome(text) that checks whether
# a string is a palindrome (ignore spaces and case) and explain it line by line for beginners
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
# user input manner
try:
    number = int(input("Enter a number to check if it is prime: "))
    if is_prime(number):
        print(f"{number} is a prime number.")
    else:
        print(f"{number} is not a prime number.")
except ValueError:
    print("Invalid input. Please enter a valid integer.")
```

Output:-

```
Enter a number to check if it is prime: 29
29 is a prime number.
```

Justification:-

The function checks if a number is prime by first handling numbers less than 2 (which are not prime). Then it iterates from 2 up to the square root of the number, checking for divisibility. If any divisor is found, it returns False; otherwise, it returns True.